

Knights of Old — Handoff Plan

Oct 2, 2026 · @Mason

Locked decisions

These came out of two concept reviews (30 questions in total) and override anything below that seems to disagree.

Topic	Decision
Combat	Hidden health, no health bar (Stick Fight style). Damage shows as paint smears and splatters building up on the fighter.
Winning	Knockoffs are the main way to win; falling off the map is an instant kill.
Unarmed attack	Era-themed fists and kicks.
Weapons	Delivery depends on the arena (see the second review below). Earlier eras' weapons vanish at each era change.
Body	Rigid head and torso, jointed arms, animated legs. No full active ragdoll in the MVP.
Audience	Online friends first; local play also supported.
Netcode	Server-authoritative (Colyseus). A small hosting bill (about \$5-20 a month) is accepted. Built in Phase 3.
Arenas	Small single-screen platforms with open void edges.
Match length	Host picks 10, 20, or 30 fights; default 30. Every length covers all six eras.
Drop-in	Friends can join at the start of the next fight. A disconnect forfeits that fight.

Topic	Decision
Art	AI-generated, then cleaned up by the owner. Disclosed on Steam.
Tone	Slapstick with mild cartoon blood, drawn as red paint. The Steam content survey will reflect mild violence.
Business	Free web demo with reduced content; paid Steam release.
Owner	Beginner relying on Claude Code, so steps must be explicit.
Spoils (losers pick first)	Removed. An optional catch-up rule (first drop lands nearer the trailing player) can be tested behind a config flag.

Second review (15 more questions)

Topic	Decision
Weapon control	Free aim: mouse or right stick sets the aim angle and the arm physically follows, so body momentum adds to swings.
Movement	Single jump, dash or roll, wall slide and wall jump. No double jump. Arenas include walls.
Costumes	The fighter's costume changes every era; the body stays the same minimal shape. About 6 costume sets.
Weapon delivery	Set per arena: start armed, sky drops, or fixed spots. Fighters without a weapon use era-themed fists and kicks.
Stalling	No timer or shrinking arena at launch. Watch playtests for hiding; if it happens, add a soft late-fight hazard behind a config flag.
Scoring	1 point to the fight winner only.

Topic	Decision
Solo and bots	Practice dummies only. Bots are out of scope for now.
Hazards	Static hazards plus one scripted event per arena.
Platforms	Desktop browsers and Steam (Windows, plus Steam Deck). No touch or phone support.
Computer	Windows. Commands should work in PowerShell; Steam Deck testing (via Proton) covers Linux.
Plan and credits	Claude subscription with no extra charges. Fable is a last resort; Opus 5.5 at <code>xhigh</code> is the top everyday setting.
Timeline	5-10 hours a week with no fixed deadline; plan for roughly 12-18 months.
Audio	Licensed royalty-free packs for sound effects; AI or licensed music. Log every license and any AI audio in <code>ASSETS.md</code> .
Settings at launch	Music and effects volume sliders, and key and button rebinding. Screen-shake toggle and colorblind markers come later.
Success	A group of 4-8 friends playtests weekly; success means they ask for another round.

Vision and pitch

Knights of Old is a physics-based party brawler where 2-4 friends fight last-man-standing rounds while the world advances through history, from sticks to swords to guns to lasers, across a ~30-fight match. It runs in the browser first (share a link, enter a room code, play) and is built from day one so it can be wrapped and shipped on Steam later.

The 10-second clip: two minimalist, oil-painted fighters flail ragdoll limbs; a caveman's club launches a rival in a cartwheel across a painted cliff, paint splattering where he lands, and the era banner flips to *Knights*.

Who it's for: friend groups on a Discord call, sessions of 15-25 minutes, streamers who want chaotic clips.

Inspirations (format only, never assets or names): Stick Fight: The Game for the weapon-and-ragdoll chaos, Rounds for the escalating, snowballing rule changes between fights.

Goals

- Free browser version as a playable demo and marketing funnel; richer Steam release afterward.
- A locked 60 fps on a mid-range laptop, with instant, responsive controls.
- Heavy, satisfying weapon impacts that are the main source of fun.

Non-goals for the MVP: accounts, matchmaking, 3D, map editor, progression systems, cosmetics, mobile.

Design pillars

Every feature gets checked against these. If it fails one, it waits.

1. **Physics is the comedy.** Hits move bodies. Impact, knockback, and flailing matter more than damage numbers.
2. **Readable at a glance.** Minimal characters with strong silhouettes; you can always tell who is who and what is dangerous.
3. **Smooth before pretty.** 60 fps and low input latency beat visual effects. Cut an effect before dropping a frame.
4. **Each era resets the rules.** New era means new weapons, new movement feel, and a new arena mood, so the match never gets stale.
5. **One session, one story.** ~30 fights build to a crowning moment, then the whole thing recycles.
6. **Instant to join.** A link and a room code. No accounts, no installs, no tutorials longer than ten seconds.

Match structure and core loop

A match (call it a *Reign*) defaults to 30 fights across 6 eras (5 fights per era), ending with a crowned champion. The host can instead pick 10 or 20 fights; era lengths scale so every Reign still covers all six eras.

A fight

- 2-4 players spawn in a single-screen arena following the arena's weapon rule (some arenas start fighters armed, others drop weapons from the sky or place them at fixed spots); a fighter without a weapon uses era-themed fists and kicks.
- Last player alive wins the fight and earns 1 point. Target length: 30-60 seconds.
- Deaths are ragdoll collapses. Eliminated players watch, and optionally throw era-themed nuisances (crows, arrows) from the sidelines.
- Environmental kills (falls, hazards) count, and are encouraged.

Between fights (about 5 seconds)

- Quick scoreboard flash and the next arena variant.
- **Catch-up drops (optional):** in arenas that use drops, the first drop of the next fight can land nearer the trailing player. Keep this behind a config flag so it can be tuned or cut.

Era progression

- Fights 1-5 Stone Age, 6-10 Antiquity, 11-15 Medieval, 16-20 Gunpowder, 21-25 Modern, 26-30 Space. See the Eras table.
- The era change is a short, punchy transition: banner and palette shift. Earlier eras' weapons vanish; only the new era's weapons spawn.

Crowning

- After fight 30, the player with the most points is crowned. A tie goes to a sudden-death fight with every era's weapons mixed.
- A victory screen shows stats (most knockbacks, longest ragdoll flight, most paint splattered).
- Then the Reign recycles: back to Stone Age, crown tally kept in the lobby.

Arenas: one theme per era, 2-3 small single-screen layouts each, with open void edges so knockoffs are always a threat, plus some walls for wall jumps. Each layout sets its own weapon delivery rule, static hazards, and one scripted event. No scrolling camera.

Eras and weapons

Six eras of five fights each. Stone Age, Medieval, Gunpowder, and Space come from the original concept; Antiquity and Modern are suggested fillers to reach 30 fights and can be swapped. Aim for 3 primary weapons per era in the MVP, plus 2-3 pickups. How weapons reach players is set per arena: a starting loadout, sky drops, or fixed spots.

Era	Fights	Primary weapons	Pickups	Movement or arena gimmick
Stone Age	1-5	Stick, bone club, thrown rock	Torch, bola	Heavy gravity, short jumps, tar pits
Antiquity	6-10	Spear, gladius, round shield	Javelin bundle, oil jar	Slippery marble, falling columns
Medieval	11-15	Longsword, mace, lance	Kite shield, crossbow	Castle walls, swinging chandeliers
Gunpowder	16-20	Musket, saber, cannon (stationary)	Powder keg, smoke bomb	Smoke clouds, ship decks that tilt
Modern	21-25	Rifle, shotgun, grenade	Jetpack burst, riot shield	Cars and crates to hide behind
Space	26-30	Laser blade, blaster, gravity gun	Grapple beam, jump boots	Low gravity, floating platforms

Weapon design rules

- Every weapon is a physical object: it has mass, length, and a grip point, and it is positioned by the player's body (see Characters, physics and controls).
- Melee is the soul of the game. Ranged weapons exist but are slower, have limited ammo, and fire projectiles with real knockback so they do not dominate.
- Weapons should differ in *feel*, not only in stats: a club is heavy and slow to swing but launches opponents; a spear is long and rewards spacing; a laser blade is light and fast.
- Each weapon is one data file (mass, length, grip offset, impact multiplier, swing penalties, sound, particle style). Adding a weapon should never require touching engine code.

Characters, physics and controls

Decision: use a real 2D rigid-body engine, **Rapier2D** (WASM), rather than hand-rolled physics. Weapons held through joints and flailing limbs need it. Prefer Rapier's deterministic build ([@dimforge/rapier2d-deterministic-compat](https://github.com/dimforge/rapier2d-deterministic-compat) ; verify the package name and version at install) so online play stays possible.

Character body (minimal, painted)

- Rigid head and torso with two jointed arms (two segments each); legs are animated, not physics-driven. Joints are springs with angle limits.
- Full active-ragdoll legs are out of scope for the MVP. They are the biggest tuning and netcode risk; revisit only after the game is fun and stable online.
- A light balance controller keeps the character mostly upright while alive. On death or heavy impact it goes fully limp.

Controls (twin-stick model everywhere)

- Move left/right, jump, dash or roll, wall slide and wall jump, aim direction, attack/swing, grab or throw, and a context action (block or alt-fire).
- Gamepad: left stick move, right stick aim, trigger attack, face button jump.
- Keyboard and mouse: WASD, mouse aim, click attack. Local 2-player on one keyboard uses a second key layout (arrows plus a cluster) with aim set by keys. Gamepads are the preferred local setup.
- All inputs reduce to one struct per player per frame, so keyboard, gamepad, network, and replays all feed the same simulation.

How weapons work

- The weapon is a rigid body joined to the hand. Aim direction pulls the arm toward that angle with spring force, and body momentum adds to swing speed. Players position weapons by moving their whole body.
- Damage and knockback come from impact: relative velocity along the contact normal times effective mass. Below a threshold nothing happens, so resting contact never hurts.
- Hits trigger hit-stop (3-6 frames), screen shake scaled to impact, a paint splat, and a weapon-specific sound.
- Weapon clashes bounce both weapons and briefly stagger both players.
- Health is hidden (no health bar): about 100 hidden HP per fighter per fight, heavy hits do 30-60. Damage shows as paint smears, splatters, and cracks building up on the body. Falling off the map is an instant kill, and knockoffs are the main way to win.

Tuning: every number above lives in one config file with hot reload and a debug overlay (frame time, body count, input state, last impact value). Claude cannot judge feel; humans tune it, so expose knobs generously.

Art direction

Look: minimalist characters (simple shapes, no faces beyond a mark or two) rendered as if painted in oil on canvas: visible brush strokes, rich color, soft edges, a faint canvas grain. Backgrounds are lush painted landscapes; characters are the simplest, boldest thing on screen so they always read.

How to get it without wrecking frame rate

- **Pre-painted assets, not a live filter.** Build fighter body parts as simple shapes filled with one painted brush texture that code tints per player color, and make costume props and weapons transparent painted sprites. Sprites attach to physics bodies. This costs almost nothing at runtime.
- **Baked painted backgrounds.** One painting per era, split into 2-3 parallax layers. Static, so any painterly effect is paid for once, offline.
- **Cheap global finish only.** A canvas-grain overlay, subtle vignette, and a slight color grade per era. Avoid full-screen painterly shaders (Kuwahara and similar); test one on a budget GPU and only keep it if it holds 60 fps.
- **Paint splatters, including mild cartoon blood drawn as red paint.** Impacts stamp brush-splat decals into a persistent render texture, so each fight leaves a colorful record on the arena. This is the signature visual and is cheap to draw.
- **Per-era palettes:** earthy ochres (Stone), marble and terracotta (Antiquity), deep reds and stone greys (Medieval), smoke and navy (Gunpowder), desaturated steel (Modern), neon on indigo (Space). Player colors must stay distinct against every palette. Each era also has its own costume set for the fighters (caveman, gladiator, knight, soldier, trooper, space ranger): the body shape stays the same and only the painted overlay changes, so budget about 6 costume sets.

Pipeline and rules

- Write a one-page style guide first (palette, stroke width, shape language, reference paintings). All art is AI-generated and then cleaned up by the owner: generate every asset against the same style guide and reference set, then fix transparency, edges, and separated body parts in an image editor.
- Track the source of every asset in `ASSETS.md` (hand-made, AI-generated with tool name, or licensed). Steam requires AI-content disclosure on the store page, and this log makes that painless.
- Don't imitate the look of Stick Fight or Rounds; the oil-painting treatment is the differentiator.

Visual style bible (suggested, confirm with first test paintings)

North star: a moving oil painting you could hang on a wall, with fighters as bold, simple brushstroke figures. Rule of thumb: a fighter is three shapes (head, torso, limbs) plus a costume, and nothing else.

Character rules

- Silhouette: round head, tall rounded torso, tapered arm and leg strokes. No fingers. Face is two dabs for eyes at most.
- Color: one flat player color per fighter with a lighter lit side and a darker shadow side painted in. The era costume adds that era's colors and one signature prop shape.
- Edge: a loose, broken dark-umber brush outline 2-3 px thick at 1080p, varying in width so it reads as painted. It keeps fighters readable against busy backgrounds and paint splatter.
- Size: a fighter is about 120 px tall on a 1920x1080 reference screen.
- Player colors (suggested, must stay distinct in every era): red `#D94A3A` , blue `#2F6FB5` , yellow `#E8B931` , green `#3E9B5A` . Add a painted chest mark (circle, triangle, square, diamond) later for colorblind players.

Era palettes and costumes (suggested starting points)

Era	Setting and light	Key colors	Costume prop
Stone Age	Cave mouth and savanna, low golden sun, long shadows	Ochre <code>#C98B3F</code> , sienna <code>#8C4A2F</code> , bone <code>#E9DDC1</code> , umber <code>#3A2618</code>	Fur wrap, bone necklace
Antiquity	Marble temple, hard bright noon	Terracotta <code>#C4623A</code> , marble <code>#EDE8DB</code> , sea blue <code>#3C7FA8</code> , olive <code>#7A8449</code>	Laurel wreath, tunic trim
Medieval	Castle courtyard, overcast with torch glow	Stone grey <code>#6B6F76</code> , deep red <code>#8E2B2B</code> , moss <code>#4F6B3F</code> , pewter <code>#A7ADB5</code>	Plumed helm, tabard
Gunpowder	Smoky battlefield at dusk	Navy <code>#25324A</code> , smoke <code>#B9B2A5</code> , brass <code>#B48A3C</code> , crimson <code>#A5342E</code>	Tricorn hat, cross-belt

Era	Setting and light	Key colors	Costume prop
Modern	Rainy industrial ruins, cold light	Steel #5C6670 , olive drab #5E6B3A , rust #A0522D , haze #8A97A0	Helmet, vest
Space	Orbital station, neon rim light	Indigo #1E1B4B , cyan #3DE0E8 , magenta #D63DAF , white-hot #F4F1FF	Visor helmet, glowing seams

Brushwork and texture rules

- Visible directional strokes that follow the form, thick impasto highlights on lit edges, and a canvas-weave overlay at about 8-12% opacity.
- No smooth airbrushed gradients, no photorealism, no glossy 3D look, and no text baked into artwork.
- For prompt vocabulary use movement terms (impasto, palette-knife texture, Post-Impressionist color) and public-domain old masters for lighting and brushwork. Avoid naming living artists or copyrighted game and film styles in prompts.

Hits and effects (all painted)

- Impacts stamp a splat in the attacker's color; mild blood is red paint. Cap at about 400 decals and fade older ones to 40%.
- Swing trails are painted arc strokes that fade over about 6 frames. Dash is a streak stroke. Dust is smudged strokes.
- A knockoff leaves a paint streak off the screen edge. Death bursts the fighter into paint drips.
- Era banners are big brush lettering that sweeps across the screen for about 1.5 seconds.

UI style: canvas-textured panels, a brushy display font for titles and one clean sans-serif for body text. Use fonts with an open license (for example from Google Fonts) and log them in `ASSETS.md`. Scoreboard bars are paint strokes.

Build approach (most beginner-friendly)

- Fighter bodies are shapes drawn in code, filled with one grayscale painted brush texture that code tints per player color. This avoids asking AI to produce clean separated limbs.

- Costume props, weapons, backgrounds, and splat decals are AI-generated painted images, cleaned up by the owner.
- Weapons are drawn horizontally with the grip near the left or center, plus a small data entry giving the grip point.

AI generation workflow

1. **Style anchors first.** Generate 20-30 candidates (a fighter, a prop, a background) and pick 3. Use these as the style reference for every later generation and keep them in `art/anchors/`.
2. **Lock a master prompt** and reuse it. Record tool, version, prompt, and seed per asset in `ASSETS.md`.
3. **Check every asset** against the anchors at actual game size before it enters the game (checklist below).
4. **Clean up** transparency, edges, and stray artifacts in an image editor; upscale backgrounds if the tool outputs small images.

MASTER PROMPT (edit the bracketed parts per asset)
[SUBJECT], simple bold shapes, oil painting on canvas, thick impasto brush strokes,
visible palette-knife texture, limited palette of [ERA COLORS], flat simplified forms,
hand-painted, loose brushed edges, no outlines other than brushed edges,
[plain flat gray background for sprites | full scene for backgrounds]
NEGATIVE (if the tool supports it): photorealistic, 3D render, glossy, airbrushed
gradients, text, watermark, logo, extra limbs, anime, pixel art, comic outlines

Asset specs (suggested)

Asset	Size	Format and notes
Arena background	3 layers (far, mid, near), 2560x1440 each	WebP at about quality 85; far layer has no transparency
Platform and wall textures	512x512, tileable	PNG; edge strips for platform tops
Fighter paint texture	512x512 grayscale, tileable	One texture, tinted in code per player color

Asset	Size	Format and notes
Costume props	256-512 px, transparent	3-5 props per era; PNG
Weapons	512 px on the long edge, transparent	PNG plus grip point in its data file
Splat decals	12-20 at 256x256, transparent	White or gray; tinted in code
UI panels and logo	9-slice panels, one logo	PNG

- Pack everything except backgrounds into at most two texture atlases of 4096x4096.
- Name files `era_type_name_variant.png`, for example `stone_weapon_club_01.png`.
- Budgets (approximate, adjust after testing): first playable web load under about 25 MB with later eras loaded on demand, and decoded textures under about 200 MB.

Style acceptance checklist (every asset)

- Readable at 120 px fighter height and against paint splatter?
- Uses its era palette and the shared brushwork look, with visible brush texture?
- No baked text, watermark, or smooth gradient?
- Source, tool, and license (or AI note) logged in `ASSETS.md`?

Owner decisions needed: pick the 3 style anchors, approve or change the era palettes, and choose the two fonts.

Tech stack and architecture

Stack

- **Language and build:** TypeScript (strict) with Vite.
- **Rendering:** PixiJS v8 (WebGL, lean, good for sprites, render textures, and light shaders). Phaser is an acceptable fallback, but its built-in physics would go unused.
- **Physics:** Rapier2D (WASM), deterministic build.
- **Audio:** WebAudio via Howler.js. Audio starts on first user gesture, so the title screen needs a click.
- **Tests:** Vitest for simulation logic, Playwright for a browser smoke test later.
- **Hosting (web):** Cloudflare Pages or Netlify for the static build; a small Node server on Fly.io or Railway when online arrives.

The one architectural rule: the simulation knows nothing about the browser.

```
src/
  sim/          pure game logic: world, fighters, weapons, rules, step(state,
inputs)         inputs)
                no DOM, no PIXI, no Math.random, no Date.now
  content/      data only: eras.ts, weapons/*.ts, pickups/*.ts, arenas/*.ts,
tuning.ts
  render/       PIXI scene, sprites, splat layer, camera shake, era transitions
input/          keyboard, gamepad, (later) network -> PlayerInput structs
audio/          sound manager and music director
net/            (later) lobby, rollback or server client
platform/       Platform interface; web.ts now, steam.ts later (saves,
achievements, lobbies)
ui/             menus, lobby, scoreboard, crown screen
main.ts         wires it together and owns the game loop
```

Game loop: fixed 60 Hz simulation step with an accumulator; rendering runs at display refresh and interpolates between the last two sim states. This keeps physics stable and online play possible.

Determinism: a seeded PRNG inside the sim, inputs as plain structs, no wall-clock reads. A test runs 1,000 frames twice from the same seed and inputs and asserts identical state hashes.

Performance budget (target: 60 fps on a 2020 integrated-GPU laptop and Steam Deck at 800p)

Item	Budget
Frame time total	16.6 ms
Simulation step	4 ms or less
Render	6 ms or less
Dynamic bodies	about 60 (4 players at about 10 each, plus projectiles and debris)
Draw calls	100 or fewer, using texture atlases
Allocation in the loop	none; pool projectiles, particles, and decals

A built-in FPS and frame-time overlay (toggle with F3) is required from the first commit. Check it after every feature.

Multiplayer and netcode

Online physics fighting is the hardest technical problem in this project. The plan is to make it *possible* from day one and *real* only after the game is fun locally.

Stage 1: local play (Phase 2). 2-4 players on one screen with gamepads and keyboard. Local play stays supported as a secondary mode, using the same simulation and the same input structs.

Stage 2: online play (Phase 3, pulled forward because online friends are the main audience). Decision: server-authoritative, with rollback kept as a possible later upgrade because the simulation stays deterministic.

Approach	How it works	Pros	Cons
Server-authoritative	Node server (Colyseus) runs the same sim; clients send inputs, receive snapshots at 20-30 Hz, interpolate	Simple to reason about, no desync, cheats are hard	Hosting cost, floaty feel without prediction, ragdoll prediction is tricky
Rollback over WebRTC	Peers exchange inputs, each runs the deterministic sim, rewind and resimulate on late inputs using Rapier world snapshots	Best feel, near-zero server cost (signaling only)	Needs strict determinism, harder to debug, desyncs across browsers

Chosen approach: server-authoritative, the safer path to a shippable product for a beginner-led project, at roughly \$5-20 a month on Fly.io or Railway.

Requirements either way

- Room codes (4 letters), max 4 players, host starts the Reign. No accounts.
- Lobby shows player colors and ready state; a disconnect forfeits that fight (the player ragdolls out) and they can rejoin at the next fight. Friends can also join mid-Reign at the next fight, starting with zero points.

- Latency tolerance matters more than exact simulation: slower projectiles, forgiving hit windows, and generous knockback keep the game fun at 80-120 ms.
- On Steam, swap the transport for Steam lobbies and Steam Networking behind the same interface.

Path to Steam

The web build is the product's first form; Steam is a wrapper plus a few integrations. Nothing in `sim/` changes for Steam.

Packaging

- Wrap the Vite build in **Electron** (consistent Chromium and WebGL on every OS, which matters for a game). Tauri is lighter but uses system webviews that vary in WebGL quality, so avoid it for this game.
- Use **Steamworks.js** for achievements, Steam Cloud, lobbies, rich presence, and the overlay. Steam overlay support inside Electron needs extra setup; test it in the first Steam spike, not at the end.
- Package with electron-builder for Windows first, then Linux (also covers Steam Deck via native or Proton).
- Put all platform calls behind the `Platform` interface (`saveData` , `unlockAchievement` , `createLobby` , `openUrl`). The web implementation uses `localStorage`; the Steam implementation calls `Steamworks.js`.

Steam Deck and controllers

- Gamepad is a first-class input from the start, not an afterthought.
- Target 60 fps at 800p, readable text at Deck size, and full navigation of menus by controller.
- Support Steam Input and show correct controller glyphs.

Store requirements and lead time

- Steamworks partner signup, \$100 app fee per game, and tax and banking setup.
- Store page must be live for at least two weeks ("Coming soon") before release, and both the page and the build go through Valve review. Budget about a month of lead time.
- Needed assets: capsule art in several sizes, 5+ screenshots, a short trailer, description, and the content survey.

- **AI disclosure:** the content survey asks about AI-generated art, audio, and text. Use the `ASSETS.md` log to answer accurately.
- Check that "Knights of Old" is free to use: search Steam, trademark databases, and the web before spending on art and branding.

Go-to-market sketch: free web version first with a Steam wishlist link, a free demo and Steam Next Fest appearance, and short clips (TikTok, Shorts) of the funniest impacts. The Steam version adds more weapons, arenas, online via Steam, and achievements, so the web version does not cannibalize it.

Roadmap

Total estimate for a beginner working part-time with Claude Code: roughly 12-18 months to a Steam release at 5-10 hours a week with no fixed deadline. Each phase ends with a gate; do not start the next phase until the gate passes.

Phase	Goal	Deliverables	Exit gate
0. Setup (3-5 days)	Foundation	Install Node, Git, and an editor; repo, Vite, Pixi, Rapier, fixed-step loop, FPS overlay, deploy to a public URL	A physics box tumbling at 60 fps on the live URL
1. Body and weapon feel (2-3 weeks)	Fun alone	One placeholder fighter with jointed arms, a stick weapon, training dummy, hit-stop, shake, sounds, and a style test (one tinted-shape fighter over one painted background) to confirm the look and frame rate	Swinging the stick at a dummy is fun for 5 minutes; 60 fps with 4 fighters

Phase	Goal	Deliverables	Exit gate
2. Local fight (3-4 weeks)	A real fight	2-4 players, gamepad and keyboard, era-themed fists and kicks, per-arena weapon delivery (starting loadout, drops, or fixed spots), hidden health, dash or roll, wall jumps, void-edge arena with knockoffs, round win	Friends ask for a rematch
3. Online (5-8 weeks)	Play remotely	Colyseus server, room codes, lobby, snapshot interpolation, disconnect and rejoin rules, hosting on Fly.io or Railway	Four players on real home connections finish 10 fights
4. Era system (3 weeks)	The hook	Weapon and era data schema, 3 eras (Stone, Medieval, Gunpowder), per-arena weapon delivery rules, host-chosen 10/20/30 fights, scoreboard, crown screen	A full Reign completes online without bugs
5. Art and audio (3-5 weeks)	The look	Style guide, AI-generated painted assets cleaned up by hand, costume sets for the first 3 eras, paint splatter and damage layer, licensed SFX, music	A stranger describes it as "painted" from a screenshot; still 60 fps
6. Content and balance (3 weeks)	All 6 eras	Remaining eras and their costume sets, 3 weapons each, arena layouts, tuning pass	Every weapon is chosen and beaten in playtests
7. Web launch (2 weeks)	Funnel	Landing page, reduced-content free demo, analytics, shareable clips, Steam wishlist link	Public URL live, first outside playtesters

Phase	Goal	Deliverables	Exit gate
8. Steam (6-10 weeks)	Release	Electron, Steamworks.js, Deck pass, store page, demo, Next Fest	Store page approved, build approved, launch

Working agreement for Claude Code

Copy the essentials of this section into the repo's `CLAUDE.md` so every session starts with them.

The owner is a beginner. Claude Code should:

- Explain each step in plain language and give exact terminal commands, never assuming Git, Node, or terminal knowledge.
- Commit to Git after every working slice with a clear message, and explain how to undo a bad change.
- After building something, run it and tell the owner exactly what to look for and what to click.
- Stop at every roadmap gate and wait for the owner to confirm before moving on.
- Prefer fixing errors itself and summarizing the cause in one or two sentences.

How to work

- Keep the game playable at all times. Work in small vertical slices and commit after each one that runs.
- Stay inside the current roadmap phase. Do not add features from later phases, and ask before adding any dependency not listed in this doc.
- Before coding a feature, state the plan in a few lines; after coding, run it and report what you saw (FPS, console errors, behavior).
- Prefer small files and clear module boundaries so changes stay local.

Hard rules for `sim/`

- No DOM, Pixi, `Math.random`, `Date.now`, or `performance.now` inside the simulation. Use the seeded PRNG and the frame counter.
- Player input enters only as a `PlayerInput` struct.
- All gameplay numbers come from `content/tuning.ts` or content data files, never magic numbers in logic.

- New weapons, pickups, eras, and arenas are data files plus assets, not engine changes.

Quality gates

- Vitest: a determinism test (same seed and inputs, 1,000 frames, identical state hash), plus unit tests for damage, knockback, scoring, and era progression.
- After every feature, check the F3 overlay: no regression below 60 fps, no per-frame allocation growth.
- Run in Chrome, Firefox, and Safari before closing a phase.

Records to maintain

- `DESIGN.md` (this doc, kept current), `DECISIONS.md` (one line per decision with date and reason), `ASSETS.md` (every asset's source and tool, for Steam AI disclosure), and `PLAYTEST.md` (what friends said, what was changed).

Where humans are required

- Judging game feel. Claude should expose tuning variants behind flags and describe how to test them rather than declare something "feels good."
- Final calls on art, audio, and balance.

Model and effort guide

Use the cheapest setup that does the job, and move up when the work gets hard. Claude Code cannot change its own model or effort; you do it with `/model` and `/effort`. The table below is based on Claude Code's current model aliases and effort levels (checked October 2026; aliases update over time, so run `/model` to see what is offered to you).

Work	Model	Effort	Move up when
Setup, installs, Git, config	Sonnet 5.5 (<code>sonnet</code>)	medium	The same error survives 2 fixes
Tuning numbers (change a value, test it)	Sonnet 5.5	low	Behavior is wrong, not just badly tuned
Weapon and joint physics, determinism design	Opus 5.5 (<code>opus</code>)	high	Use <code>opusplan</code> so Opus plans and Sonnet builds

Work	Model	Effort	Move up when
Local gameplay (hidden health, weapon delivery, rounds, knockoffs)	Sonnet 5.5	medium	Rules interact in surprising ways
Netcode, lobby, desyncs	Opus 5.5	high, <code>xhigh</code> while debugging desyncs	The same bug survives 2 fixes: stay on Opus at <code>xhigh</code> and ask for a written root-cause analysis
Weapon, era, and arena data files; UI screens	Sonnet 5.5	low or medium	Rarely needed
Renames, typos, formatting, comments	Haiku 4.5 (<code>haiku</code>)	no effort control	Never
Big refactors, root-cause hunts, architecture decisions	Opus 5.5	<code>xhigh</code>	Split the work into steps; Fable only if you choose to accept usage credits
Electron and Steamworks packaging	Sonnet 5.5	medium	Overlay or build failures: Opus 5.5 at high

Notes

- On the Anthropic API and paid plans, the default model is Opus 5.5 at `medium` effort.
- Fable is the most capable but can bill to usage credits depending on your plan. You chose to avoid extra charges, so treat Fable as a rare last resort and decline its consent prompt unless you decide otherwise.
- `max` effort is for last-resort problems only: it can overthink and show diminishing returns.
- For a one-off deep think without changing settings, put the word `ultrathink` anywhere in your message.
- In the `/model` picker, press `s` to switch for this session only, so your saved default does not change.

Quick commands

/model sonnet	everyday building
/model opus	hard physics or netcode
/model opusplan	Opus plans, Sonnet builds
/model fable	rarely: may bill usage credits
/model haiku	trivial edits
/effort low	quick tweaks
/effort medium	normal feature work
/effort high	bugs and risky changes
/effort xhigh	deep debugging

Paste this into **CLAUDE.md** so Claude Code tells you when to switch

Model and effort advisor

You cannot change your own model or effort. The owner does that with /model and /effort. Your job is to say when a change would help.

Before starting a task, classify it using the "Model and effort guide" in DESIGN.md and compare it to the model and effort shown in the session header.

SWITCH UP when any of these is true:

- The same fix has failed twice, or a test still fails after three attempts.
- The task touches sim determinism, physics joints, netcode, or desyncs.
- The change spans five or more files, or is an architecture decision.
- The owner says "it still doesn't work".

Say exactly one line, then wait for the owner's reply before continuing:

SWITCH UP: <reason>. Run: /model opus then /effort high
(use /effort xhigh while debugging a desync. Never suggest /model fable: the owner avoids extra usage charges.)

SWITCH DOWN when the task is only: editing a data file, changing a tuning number, renaming, formatting, or writing comments. Say one line, then keep working:

SWITCH DOWN: <reason>. Run: /model sonnet then /effort low
(use /model haiku for trivial edits)

ONE-OFF: if a single question needs deep reasoning, say:

TIP: add the word ultrathink to your next message.

Rules: suggest at most once per task; never suggest max effort unless the owner asks; if you are unsure what model is active, ask the owner to run /status.

Risks, open questions, and first prompt

Biggest risks

Risk	Why it matters	Mitigation
Ragdoll tuning	Active ragdolls are fiddly and can look broken or feel mushy	Start with the simplest jointed body; add limbs only after it is fun; heavy tuning config
Netcode	Physics-heavy online play can eat months	Server-authoritative from Phase 3; deterministic sim from day one; cap at 4 players; test with artificial 100 ms latency
Frame rate	Painterly effects and WASM physics can stall low-end machines	Baked art, budget table, FPS overlay, test on a weak laptop and Steam Deck early
Content volume	6 eras with weapons, arenas, and art is a lot	Ship 3 eras first; each era is data plus assets
Crowded genre	Many friend-slop fighters exist	The era-hopping hook plus the painted look; test the 10-second clip early
Name clash	Another game or trademark may share the name	Search before investing in branding

Open questions (decide with playtesters)

- Is a catch-up drop rule needed so trailing players are not stuck unarmed, and how often and where do weapons drop?
- Mouse and keyboard aim versus twin-stick only: is a second local keyboard layout worth supporting?
- Max players: 4, or push to 6-8 later?
- Which AI image tool and workflow keeps one consistent oil-painting style across every asset?
- Exact Steam price, and which eras and arenas the free web demo includes.

First prompt to paste into Claude Code

Read DESIGN.md (the Knights of Old handoff plan) fully before doing anything. I am a beginner: explain each step in plain language and give exact terminal commands for Windows PowerShell. The Locked decisions section overrides anything that conflicts.

We are doing Phase 0 and Phase 1 only.

Phase 0: scaffold a TypeScript + Vite + PixiJS v8 + Rapier2D (deterministic build) project with the folder layout from the doc, a fixed 60 Hz simulation loop with render interpolation, an F3 overlay showing FPS and frame time, and a Vitest determinism test. Create CLAUDE.md from the Working agreement section, plus DECISIONS.md and ASSETS.md.

Phase 1: one fighter built from simple physics bodies (start with torso, head, and two jointed arms), one weapon (a stick) held through a joint and aimed with the mouse or right stick, a training dummy, impact-based hidden-health damage and knockback, fists as the unarmed attack, hit-stop, screen shake, and placeholder sounds. Put every number in content/tuning.ts with hot reload.

Stop at the Phase 1 exit gate and tell me what to playtest and which tuning values to try. Do not start Phase 2.